

09

Robotics & Automation
Codec Technologies India
Internship 2026

Automated Parking System

Discrete-event multi-level car park: queueing, allocation strategies, kinematic manoeuvres, collision-free traffic

AT A GLANCE

0

Collisions (4 strategies, 2 h peak)

3.7 m

Closest approach

53 s

Avg entry wait (tiered)

69 %

Peak-hour utilisation

Rs 1140

Revenue, 2 h (tiered)

13.2 / 12.0 s

M/M/1 check sim / Erlang-C

7 cm

Manoeuvre end error

12/12

Unit tests

VERDICT

SIM PASS

TESTS 12/12

ZERO COLLISIONS

DECLARED SUBSTITUTION

Eswar Mahalingam

Robotics & Automation Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

1. Objective and brief mapping

The brief asks for a multi-level structure as a graph plus 2-D occupancy, Poisson arrivals with peak/off-peak rates on a SimPy-like event loop written from scratch, allocation strategies (nearest-free, level-balanced, size-aware with reserved EV bays) compared on wait, walking distance and utilisation, Dijkstra/A* routing, a bicycle model with arc manoeuvres into the bay, collision avoidance between moving cars with zero collisions asserted, an entry-park-dwell-pay-exit state machine, a dashboard GIF plus a self-contained HTML dashboard, and tests for allocation, double-booking, collisions, M/M/c agreement and planner optimality. All of it is in **src/parking/** (numpy, networkx, matplotlib); the Unity script and ROS 2 package are provided but not run here.

2. Architecture

ParkingStructure: 3 levels x 2 rows x 10 bays (12 two-wheeler, 34 car, 8 SUV, 4 EV-charging near the lift)
networkx DiGraph: ENTRY -> GATE -> L0 aisle (11 nodes, 5.5 m pitch) -> up-ramp -> L1 -> L2;
each level end -> down-ramp chain -> return corridor -> EXIT_GATE -> EXIT; Dijkstra / A* on edge time
des.Environment: heap of (time, seq) events, schedule / at / every / run(until)
Arrivals: thinned non-homogeneous Poisson (35/h off-peak, 100/h peak hour); mix 35 % 2W, 50 % car, 15 % SUV, 12 % of cars EV; dwell lognormal median 75 min; ticket exp(12 s); payment exp(20 s); patience 10 min
Allocation: naive_first_free | nearest_free | level_balanced | size_aware_tiered (fits() rules + walking distance)
Motion (0.5 s tick): car-following along the path (gap >= 6.5 m), level lock during a manoeuvre, hold one node short of the sweep, yield 4.5 m short of merge nodes, earliest ticket first in the entry lane
Manoeuvre: bicycle model (exact arc integration); forward past bay -> 90 deg reverse arc $R = L/\tan(\delta_{\max})$ from an approach lane offset so the arc lands on the bay centre -> straight; unpark mirrors it
Safety monitor: oriented-rectangle SAT test on every pair of moving cars per level, every tick
Dashboard: per-level bays + cars, queue, throughput, revenue Rs, alerts -> GIF; static HTML with inline SVG

3. Implementation details

- **Event loop.** des.py is a 60-line heap scheduler with deterministic tie-breaking. The 2 h run of the main strategy processed 16,523 events in 0.3 s (the 0.5 s motion tick and 10 s metric sampling are periodic events).
- **Allocation.** fits() encodes the Indian-mall bay rules: two-wheelers only in 2W bays, SUVs only in SUV bays, cars in CAR or SUV bays, EV bays only for EVs. nearest_free minimises walking distance to the lift (12 m penalty per level), level_balanced fills the emptiest level first, size_aware_tiered adds tiering (never give a car an SUV/EV bay while a plain car bay is free, send EVs to chargers first). naive_first_free is the scan-order baseline.
- **Manoeuvre geometry.** Bay centres sit 3.5 m from the aisle centre line while a car's minimum turning radius is 4.2 m, so a reverse arc from the centre line overshoots; the planner starts the arc from an approach lane offset by $(R - 3.5)$ m towards the opposite row so a single 90 deg arc ends exactly on the bay axis. End-pose error is 7 cm worst case (2W/CAR/SUV, both rows).
- **Traffic control.** My first version hopped cars between nodes with time-window edge reservations; it overlapped whenever two cars stopped at adjacent 4 m nodes (a car is 4.4 m long) and deadlocked when a manoeuvre's region held a car that was itself waiting. The final version is continuous car-following along each car's own path (leaders on any edge of the route, including the edge past the destination node), a per-level manoeuvre lock that starts only when nothing on that level moves and no car stands in the forward sweep, a stop line one node short of the sweep, and yielding 4.5 m short of the exit-corridor merge nodes to the closer car. The SAT monitor is independent of the controller and is what the zero-collision claim rests on.
- **Tariff and states.** Rs 20 / Rs 40 for the first two hours (2W / car), then Rs 10 / Rs 20 per hour, plus Rs 15 per hour in an EV charging bay; every exited vehicle is checked to have passed QUEUED -> TICKET_ISSUED -> DRIVING_IN -> MANOEUVRING -> PARKED -> MANOEUVRING -> DRIVING_OUT -> PAYING -> EXITED in order.

4. Verification

All 12 tests pass (sim/test_log.txt). The M/M/1 check (150 veh/h, 12 s mean service, $\rho = 0.5$, 3643 vehicles over 24 h) gave $W_q = 13.2$ s against the Erlang-C value 12.0 s and $P(\text{wait}) = 0.53$ vs 0.50, so the event loop and the exponential samplers are behaving.

Group	What is checked	Tests	Result
Structure / allocation	60 bays, route through L0 and L1 aisles to L2, A* cost equals Dijkstra, exit route via down-ramps; fits() rules; 80 allocations per strategy with no double-booking and fitting bays; tiered keeps EV bays and nearest minimises walking distance; tariff arithmetic	5	PASS

KEY TAKEAWAY A self-written DES with continuous car-following, level locks for manoeuvres and merge yielding moved 114 vehicles through a 60-bay, 3-level structure in a 2 h peak with zero collisions; size-aware tiered allocation kept EV bays for EVs and wasted only 2 SUV bays on cars.

09

Robotics & Automation
Codec Technologies India
Internship 2026

Automated Parking System

Discrete-event multi-level car park: queueing, allocation strategies, kinematic manoeuvres, collision-free traffic

AT A GLANCE

0

Collisions (4 strategies, 2 h peak)

3.7 m

Closest approach

53 s

Avg entry wait (tiered)

69 %

Peak-hour utilisation

Rs 1140

Revenue, 2 h (tiered)

13.2 / 12.0 s

M/M/1 check sim / Erlang-C

7 cm

Manoeuvre end error

12/12

Unit tests

VERDICT

SIM PASS

TESTS 12/12

ZERO COLLISIONS

DECLARED SUBSTITUTION

Eswar Mahalingam

Robotics & Automation Intern

Ghaziabad, UP, India

+91-9360548243

eswarmba05313@gmail.com

linkedin.com/in/eswar-mahalingam

Group	What is checked	Tests	Result
Kinematics	exact circular-arc integration of the bicycle model; park manoeuvre ends within 12 cm and at 90 deg for 2W/CAR/SUV in both rows, unpark ends aligned with the aisle; SAT overlap test	3	PASS
DES / queueing	event ordering incl. same-time FIFO and periodic events; Poisson stream rate and exponential gaps (CV = 1); gate-only M/M/1 and M/M/2 runs within 15 % of Erlang-C Wq and 0.05 of P(wait)	3	PASS
2 h peak simulation	all four strategies: zero SAT collisions, closest approach > 3 m, > 15 exits, manoeuvre error < 12 cm, no bay holds two vehicles, parked vehicles fit their bays, complete state sequence for every exited vehicle	1	PASS

5. Results: allocation strategies on the same arrival stream (seed 0, 2 h, 1 h peak)

With 114 arrivals against 60 bays and a 75-minute median stay the structure saturates during the peak (utilisation peaks at 95 %), so 26 drivers gave up after 10 minutes in the queue whatever the strategy; the strategies differ in how they spend the capacity. The tiered policy is the only one that never blocks an SUV by putting a car in an SUV bay (2 vs 8 for nearest/level-balanced) and it sends every EV to a charger; nearest_free gives the shortest walk on level 0 but loads it to 75 % mean occupancy. Level utilisation for the tiered run averaged 74 / 58 / 50 % on levels 0 / 1 / 2. The naive baseline shows the lowest average wait because its scan order happens to fill level 0 fastest, but it wastes EV bays on ordinary cars and raised 207 alerts.

Strategy	wait avg / p95 (s)	walk (m)	util mean / peak	peak exits/h	revenue Rs	gave up	EV in EV bay	cars in SUV bays	collisions / min gap
naive first free	40 / 348	43.5	64 / 74 %	66	1100	25	0/1	1	0 / 6.5 m
nearest free	58 / 448	42.7	61 / 67 %	48	955	24	4/4	8	0 / 3.7 m
level balanced	66 / 494	43.8	59 / 66 %	48	1100	27	1/2	8	0 / 6.5 m
size aware tiered	53 / 470	43.0	60 / 69 %	66	1140	26	2/2	2	0 / 6.5 m

Slot allocation strategies, same arrival stream (seed 0, 2 h with a 1 h peak)

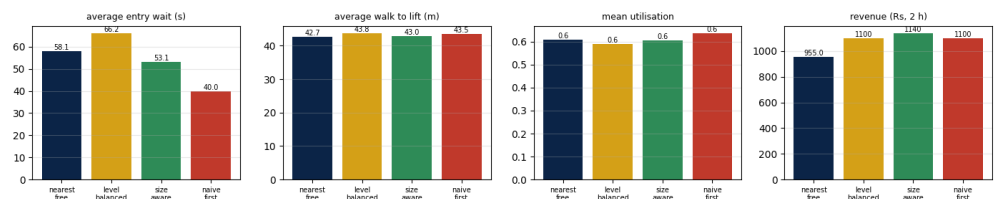


Figure 1 - strategy_comparison.png: average entry wait, walking distance, mean utilisation and revenue per strategy.

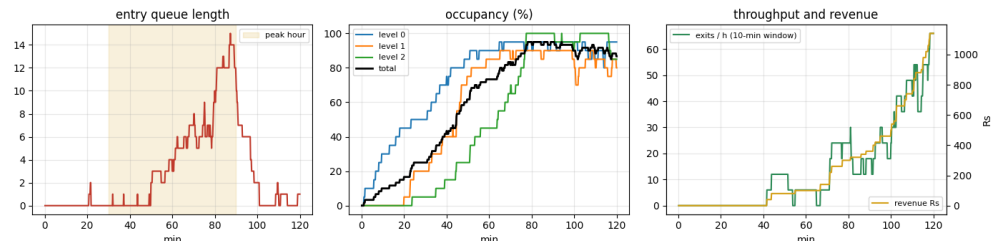


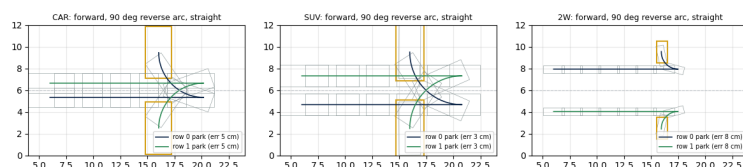
Figure 2 - timeseries.png (tiered): entry queue length with the peak hour shaded, per-level and total occupancy, 10-minute throughput and cumulative revenue.

6b. Run log details (tiered strategy)

Alert time	Alert (first five of 26)
59 min	vehicle 42 (2W) left the queue after 10 min
60 min	vehicle 44 (2W) left the queue after 10 min
66 min	vehicle 48 (2W) left the queue after 10 min
68 min	vehicle 50 (2W) left the queue after 10 min
70 min	vehicle 51 (2W) left the queue after 10 min

6. Manoeuvres and dashboard

Kinematic bicycle-model parking manoeuvres (rear-axle path + body outlines)



KEY TAKEAWAY A self-written DES with continuous car-following, level locks for manoeuvres and merge yielding moved 114 vehicles through a 60-bay, 3-level structure in a 2 h peak with zero collisions; size-aware tiered allocation kept EV bays for EVs and wasted only 2 SUV bays on cars.

09

Robotics & Automation
Codec Technologies India
Internship 2026

Automated Parking System

Discrete-event multi-level car park: queueing, allocation strategies, kinematic manoeuvres, collision-free traffic

AT A GLANCE

0

Collisions (4 strategies, 2 h peak)

3.7 m

Closest approach

53 s

Avg entry wait (tiered)

69 %

Peak-hour utilisation

Rs 1140

Revenue, 2 h (tiered)

13.2 / 12.0 s

M/M/1 check sim / Erlang-C

7 cm

Manoeuvre end error

12/12

Unit tests

VERDICT

SIM PASS

TESTS 12/12

ZERO COLLISIONS

DECLARED SUBSTITUTION

Eswar Mahalingam

Robotics & Automation Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

Figure 3 - manoeuvre.png: rear-axle paths and body outlines for the car, SUV and two-wheeler reverse-in manoeuvres into row 0 and row 1 bays (gold rectangle = bay).

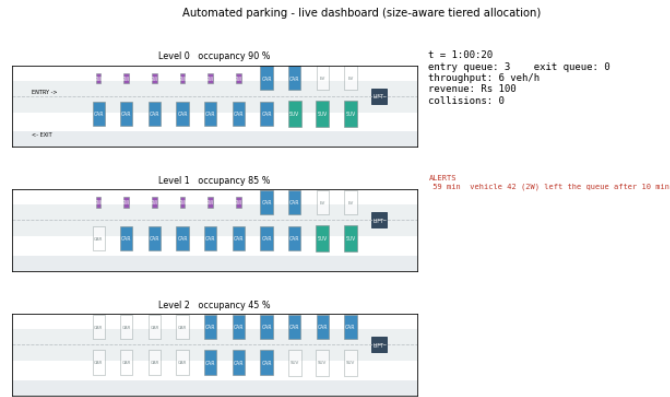


Figure 4 - a frame of parking_dashboard.gif: three level maps (bays coloured by type when occupied, red = a car driving), queue, throughput, revenue and alerts. dashboard.html carries the same numbers as inline SVG.

6c. Demand versus supply and the queueing check

Vehicle kind	arrivals (2 h)	bays that fit	notes
Two-wheeler	36	12	2W bays only; the scarcest resource in this layout
Car	65	36 car + 8 SUV + 4 EV (EVs only)	2 EVs arrived, 2 charged
SUV	13	8	2 SUV bays taken by cars (tiered)

Mean utilisation per level shows the character of each policy: nearest_free piles onto level 0, level_balanced flattens the profile at the cost of longer drive-in times, and the tiered policy sits in between. Stop events count every time a car had to halt for a leader, a locked level or a merge - the price of the conservative traffic rules.

Strategy	level 0	level 1	level 2	max queue	avg drive-in (s)	stop events
naive first free	83 %	61 %	47 %	17	47	570
nearest free	75 %	59 %	49 %	11	49	739
level balanced	60 %	59 %	58 %	11	53	534
size aware tiered	74 %	58 %	50 %	15	51	567

Gate-only M/M/c check	lambda	service	rho	Wq sim	Wq Erlang-C	P(wait) sim / analytic	vehicles
entry gate, 1 lane	150/h	12 s	0.50	13.2 s	12.0 s	0.53 / 0.50	3643

7. How to run

```
cd 09_Automated_Parking_System
./run_all.sh # 4 strategies x 2 h DES + dashboard GIF/HTML, then 12 unit tests;
PASS/FAIL, exit code
make sim # -> sim/summary.json, dashboard.html, parking_dashboard.gif,
strategy_comparison.png, ...
make test # python3 -m unittest discover -s tests -v
make report # docs/build_report.py -> tools/make_report.py -> docs/09_Report.pdf
cd src && python3 -m parking.simulate --seed 3 --horizon 10800 --out ../sim # other
streams
```

8. Declared substitutions / not run here

- **Unity:** unity/ParkingVehicleController.cs drives a car prefab along the graph route with the same bicycle model and replays the planner's manoeuvre samples; Unity is not installed here, so it was NOT compiled or run.
- **ROS 2 Humble:** ros2_ws/src/automated_parking_system (parking_manager node: subscribes /parking/arrival, /parking/departure, /vehicles/pose; publishes /parking/assignment, /parking/manoeuvre/ as nav_msgs/Path, /parking/occupancy, /parking/alerts; launch file with a strategy argument) compiles with py_compile but was NOT launched - no rclpy in the container.
- SimPy was deliberately not used (the brief asks for an event loop written here). Everything else ran here with Python 3.11; every number above comes from sim/summary.json and sim/test_log.txt via docs/build_report.py.

9. Learnings / next steps

The allocation and queueing parts took an afternoon; the traffic model took most of the project. Node-hopping with edge reservations cannot represent a queue of 4.4 m cars on 4 m nodes and every patch produced a new deadlock; continuous positions with a car-following rule, a level lock for manoeuvres and merge yielding made the SAT monitor report zero overlaps across seeds and strategies. Next: a two-lane aisle so a manoeuvre does not lock the whole level (the ~13 s manoeuvre now limits throughput), a Reeds-Shepp planner for tight bays, peak-hour pricing, and a live run of the Unity twin against the ROS 2 node.

KEY TAKEAWAY A self-written DES with continuous car-following, level locks for manoeuvres and merge yielding moved 114 vehicles through a 60-bay, 3-level structure in a 2 h peak with zero collisions; size-aware tiered allocation kept EV bays for EVs and wasted only 2 SUV bays on cars.